

Rendu individuel

Zaafir Mougammadou Zaccaria
Infrastructure cloud & déploiement applicatif

Auteur	Zaafir Mougammadou Zaccaria
Rôle	Infrastructure cloud & déploiement applicatif
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Infrastructure as code Terraform, application FastAPI/React, déploiement

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

Au sein de l'équipe, j'ai assumé la responsabilité de deux briques indissociables : l'infrastructure Azure décrite sous forme de code, et l'application elle-même, de son développement jusqu'à son déploiement sur le cluster. Mon fil conducteur a été un principe simple mais exigeant : partir d'un dépôt vide, il devait être possible de reconstruire l'intégralité de l'environnement — réseau, cluster, base de données, application — de façon entièrement automatique et strictement reproductible, sans qu'aucune étape ne repose sur une manipulation manuelle mémorisée par un membre de l'équipe.

Ce positionnement me plaçait à la charnière entre le développement et l'exploitation, ce qui correspond exactement à l'esprit DevOps : je devais comprendre les besoins fonctionnels du cabinet pour concevoir l'application, tout en anticipant les contraintes d'hébergement, de sécurité et de coût qui conditionnent sa mise en production. Cette double casquette a nourri l'ensemble de mes choix techniques, du modèle de données jusqu'à la façon d'injecter les secrets dans les conteneurs.

Concrètement, mon travail conditionnait celui des autres membres : sans infrastructure provisionnée et sans application conteneurisée, ni la chaîne d'intégration, ni la supervision, ni le plan de reprise n'avaient d'objet. J'ai donc veillé, dès le début du projet, à livrer rapidement une première version fonctionnelle afin que chacun puisse avancer sur son périmètre.

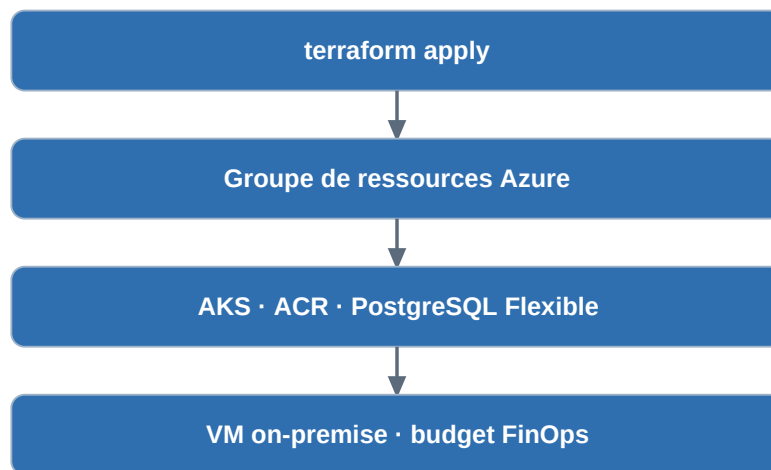
2. Ma contribution détaillée

2.1 Conception de l'infrastructure Terraform

J'ai écrit les modules Terraform qui décrivent l'ensemble des ressources Azure du projet : le groupe de ressources qui les regroupe, le cluster Kubernetes managé (AKS), le registre d'images Azure Container Registry, le serveur PostgreSQL Flexible, ainsi que la configuration réseau et le budget de surveillance des coûts. Chaque ressource est paramétrée par des variables afin de pouvoir ajuster la région, la taille des machines ou l'activation de certains composants sans modifier le code.

Un soin particulier a été porté à l'**idempotence** de cette description : appliquée plusieurs fois, elle doit converger vers le même état sans effet de bord. C'est cette propriété qui permet à la chaîne d'intégration de rejouer le déploiement en toute confiance, et qui garantit qu'un membre de l'équipe obtiendra exactement la même infrastructure qu'un autre. J'ai également veillé à ce que les dépendances entre ressources soient correctement exprimées, afin que Terraform les crée dans le bon ordre : par exemple, le registre d'images et le rôle d'accès associé doivent exister avant le cluster qui devra tirer les images.

La gestion de l'état Terraform a fait l'objet d'une attention spécifique. Plutôt qu'un état local, fragile et non partageable, nous avons opté pour un état distant stocké dans un compte de stockage Azure, ce qui permet le travail à plusieurs et évite les conflits d'écriture. J'ai conçu la configuration de sorte que ce compte de stockage soit créé automatiquement s'il n'existe pas, supprimant ainsi toute étape préalable manuelle et rendant le premier déploiement aussi simple que les suivants.



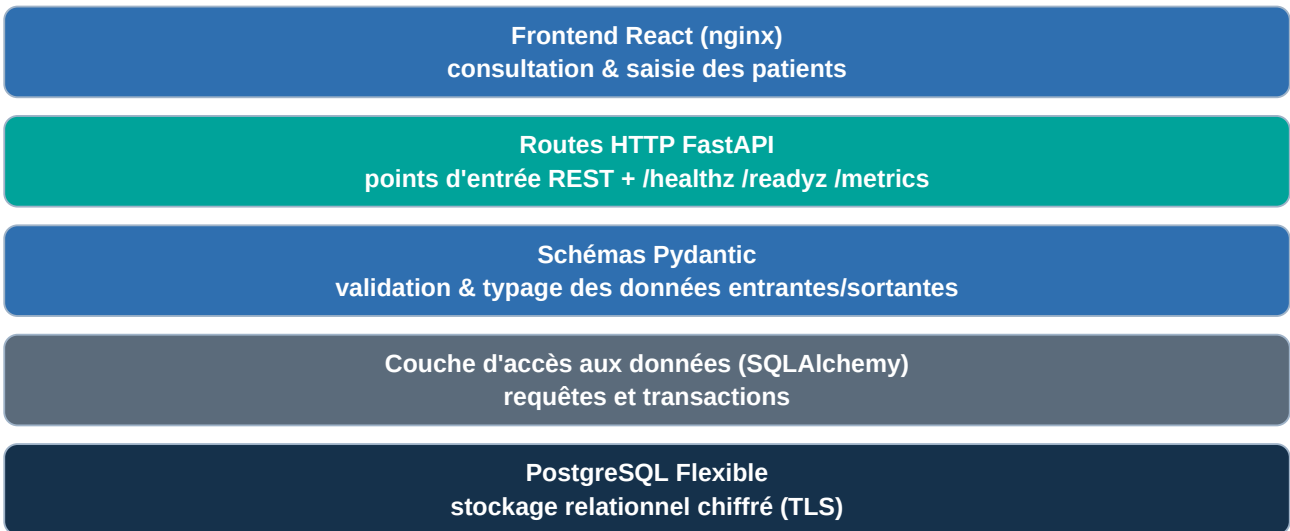
2.2 Développement de l'application

J'ai développé le cœur applicatif avec le framework FastAPI, en Python. L'API expose les ressources métier du cabinet : les patients, les appareils auditifs qui leur sont associés et les rendez-vous de suivi. J'ai structuré le code en couches claires — modèles de données, schémas de validation, logique d'accès aux données et routes HTTP — afin qu'il reste lisible et maintenable. La validation des entrées est confiée à Pydantic, ce qui garantit qu'aucune donnée mal formée n'atteint la base et que les erreurs sont renvoyées au client de façon explicite.

J'ai doté l'API de **sondes de santé** distinctes : une sonde de vivacité qui confirme que le service répond, et une sonde de disponibilité qui vérifie en outre que la base de données est joignable. Cette distinction est essentielle pour Kubernetes, qui s'appuie sur ces sondes pour décider de redémarrer un conteneur ou de lui envoyer du trafic. J'ai également exposé un point de terminaison de métriques, consommé par la supervision mise en place par Adame, ce qui illustre l'interdépendance de nos périmètres.

Côté interface, j'ai réalisé un frontend React servi par un serveur nginx, permettant au cabinet de consulter, créer et supprimer des patients de façon simple et immédiate. L'interface consomme l'API via un chemin unique, ce qui simplifie considérablement le routage en production, où le frontend et le backend sont exposés derrière le même point d'entrée. J'ai porté attention à ce que l'expérience reste fluide même en cas d'erreur réseau, en affichant des messages compréhensibles à l'utilisateur.

J'ai structuré l'application en couches nettement séparées, chacune n'ayant connaissance que de la couche immédiatement inférieure. Cette organisation, illustrée ci-dessous, rend le code lisible, testable et évolutif : on peut par exemple faire évoluer la logique d'accès aux données sans toucher aux routes HTTP.

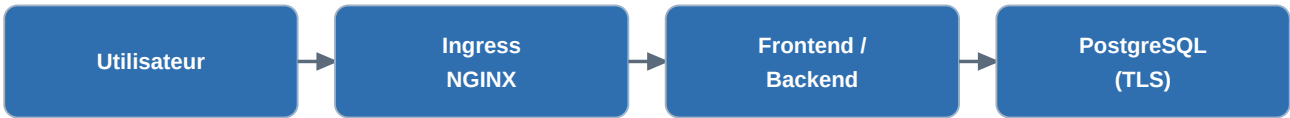


Architecture applicative en couches : chaque niveau isole une responsabilité, du rendu de l'interface jusqu'au stockage relationnel.

2.3 Conteneurisation et déploiement

J'ai conteneurisé les deux composants au moyen d'images Docker construites en plusieurs étapes (multi-stage), afin de séparer la phase de compilation de l'image finale et d'obtenir des images légères, plus rapides à distribuer et à démarrer. Ces images s'exécutent avec un utilisateur non privilégié, conformément aux bonnes pratiques de sécurité, un point sur lequel j'ai collaboré étroitement avec le responsable sécurité de l'équipe.

Le déploiement sur le cluster s'appuie sur le chart Helm conçu avec Anis : j'ai veillé à ce que l'application reçoive sa configuration — notamment la chaîne de connexion à la base — par l'intermédiaire d'un secret Kubernetes injecté au moment du déploiement, et jamais écrit en clair dans le dépôt. J'ai enfin coordonné l'enchaînement complet du déploiement de bout en bout et vérifié, après chaque livraison, le bon fonctionnement de l'application, de la base et de l'interface. Le flux de traitement d'une requête utilisateur suit le chemin suivant :



3. Choix technologiques : pourquoi ces technologies plutôt que d'autres

Chacune des briques dont j'avais la charge a fait l'objet d'une comparaison explicite avec ses alternatives. Je détaille ci-dessous les trois décisions les plus structurantes — l'outil d'infrastructure, le framework applicatif et le moteur de base de données — sous forme de tableaux comparatifs suivis de leur justification.

3.1 Infrastructure as code : Terraform plutôt que Bicep ou Pulumi

Critère	Terraform (retenu)	Bicep / ARM	Pulumi
Portabilité	Agnostique (multi-cloud)	Azure uniquement	Multi-cloud

Critère	Terraform (retenu)	Bicep / ARM	Pulumi
Langage	HCL déclaratif, lisible	DSL Azure / JSON verbeux	Langages généraux (TS, Go...)
Gestion de l'état	État distant mûr et partagé	Pas d'état (géré par Azure)	État propriétaire, récent
Écosystème / modules	Très large, communauté active	Limité à Azure	Plus restreint
Adéquation au projet	Standard du marché DevOps	Enfermement Azure	Surdimensionné ici

J'ai retenu **Terraform** parce qu'il est le standard de fait de l'infrastructure as code et qu'il reste agnostique du fournisseur : si le cabinet devait un jour migrer une partie de sa plateforme vers un autre cloud, la logique de description resterait valable. Bicep, bien qu'élégant et sans état à gérer, aurait enfermé le projet dans l'écosystème Azure ; Pulumi, qui permet de décrire l'infrastructure dans un langage de programmation classique, apportait une puissance dont nous n'avions pas besoin pour un MVP et introduisait une dépendance à un état propriétaire. La lisibilité déclarative du HCL et la maturité de la gestion d'état distant ont donc été décisives.

3.2 Framework applicatif : FastAPI plutôt que Flask ou Django

Critère	FastAPI (retenu)	Flask	Django
Validation / typage	Native via Pydantic	Manuelle ou extensions	Forms / DRF
Documentation d'API	OpenAPI générée seule	À ajouter manuellement	DRF, plus lourd
Performances	Élevées (asynchrone, ASGI)	Synchrone (WSGI)	Synchrone par défaut
Poids pour un MVP API	Léger, ciblé sur l'API	Léger mais peu structurant	Lourd (ORM, admin, templates)

Le choix de **FastAPI** s'explique par sa validation typée native, sa documentation OpenAPI générée automatiquement et sa nature asynchrone, qui offre de bonnes performances sur des machines volontairement modestes. Flask aurait exigé d'assembler manuellement validation et documentation, au risque d'une moindre rigueur ; Django, très complet, aurait apporté un ORM, une interface d'administration et un moteur de templates dont une API pure n'a pas l'usage, alourdissant inutilement l'image conteneur et la surface d'attaque. FastAPI offrait le meilleur rapport entre robustesse et sobriété pour notre besoin.

3.3 Base de données : PostgreSQL plutôt que MySQL ou MongoDB

Critère	PostgreSQL (retenu)	MySQL	MongoDB
Modèle de données	Relationnel riche	Relationnel	Documentaire
Adéquation métier	Idéal (patients ↔ appareils ↔ RDV)	Convient	Peu adapté aux relations
Offre managée Azure	Flexible Server, peu coûteux	Flexible Server	Cosmos DB, plus cher
Intégrité	ACID, contraintes fortes	ACID	Cohérence plus souple

Les données du cabinet sont fortement relationnelles : un patient possède des appareils et des rendez-vous, avec des contraintes d'intégrité qu'une base documentaire comme MongoDB gèrerait mal. **PostgreSQL** s'est imposé pour sa richesse relationnelle, son respect strict des propriétés ACID et son offre managée peu coûteuse sur Azure (Flexible Server), qui nous décharge des sauvegardes, des correctifs et de la supervision de bas niveau. MySQL aurait convenu, mais PostgreSQL offre des types et des contraintes plus riches ; MongoDB, orienté documents, aurait compliqué la modélisation de relations pourtant naturelles ici, et sa déclinaison managée Azure (Cosmos DB) est nettement plus onéreuse.

3.4 Hébergement de la base et conteneurs : arbitrages de sécurité

Concernant l'hébergement de la base, nous avons retenu un accès public restreint par un pare-feu n'autorisant que les ressources internes à Azure, avec chiffrement TLS obligatoire. C'est un compromis assumé : un réseau entièrement privé aurait été plus sûr, mais aurait ajouté une complexité — réseau virtuel dédié, résolution DNS privée, points de terminaison privés — difficilement justifiable pour un MVP au budget serré. J'ai documenté ce compromis afin qu'il soit clairement identifié comme un axe d'amélioration prioritaire pour une éventuelle mise en production réelle.

Enfin, le choix d'images Docker multi-stage et non privilégiées relève à la fois de la sécurité et de la sobriété : des images plus petites consomment moins de stockage dans le registre, se transfèrent plus vite et réduisent la surface d'attaque, autant de bénéfices alignés avec les objectifs transversaux du projet.

4. Difficultés rencontrées et solutions apportées

La principale difficulté a été d'**industrialiser le provisionnement** pour qu'il soit réellement rejouable par la chaîne d'intégration. Les premières exécutions ont révélé des situations délicates : un déploiement interrompu laissait l'état Terraform verrouillé, empêchant toute reprise ultérieure. En collaboration avec Elyess, responsable de la chaîne, nous avons ajouté un mécanisme libérant automatiquement ce verrou résiduel avant chaque application, ce qui a rendu la chaîne robuste face aux interruptions.

Une deuxième difficulté a concerné la cohérence entre la version de l'application déployée et la configuration attendue. J'ai résolu ce point en faisant en sorte que chaque déploiement utilise une étiquette d'image unique, ce qui force le remplacement propre des conteneurs et garantit que la nouvelle configuration — y compris les secrets — est bien prise en compte, sans réutilisation silencieuse d'une ancienne image mise en cache.

Un troisième problème, plus subtil, est apparu au moment de la sauvegarde de la base : un caractère spécial présent dans le mot de passe généré cassait l'interprétation de l'URL de connexion par l'outil de sauvegarde. Après diagnostic, j'ai imposé la génération d'un mot de passe strictement alphanumérique — l'entropie restant largement suffisante sur vingt-quatre caractères — ce qui a éliminé toute une classe d'erreurs d'interprétation, aussi bien pour l'application que pour les outils annexes.

Chacune de ces difficultés m'a appris qu'une infrastructure fiable ne se conçoit pas d'un seul jet : elle se durcit par itérations, au contact des cas réels, en traitant chaque incident à sa racine plutôt qu'en

le contournant ponctuellement.

5. Perspectives d'évolution

À court terme, je remplacerais l'authentification de la chaîne d'intégration par une fédération d'identité OIDC entre GitHub et Azure. Ce mécanisme substitue aux identifiants stockés des jetons éphémères délivrés à la demande, ce qui constitue l'état de l'art en matière de sécurité et lèverait l'actuelle contrainte d'un compte sans double authentification.

Je ferais également évoluer la base vers un accès strictement privé, sans aucune exposition publique, en plaçant l'application et la base dans un réseau virtuel commun relié par des points de terminaison privés. Cette évolution renforcerait significativement l'isolation des données de santé et rapprocherait l'architecture des exigences d'un véritable contexte de production soumis au RGPD.

À moyen terme, j'introduirais un outil de migrations de schéma versionnées, qui remplacerait avantageusement la création automatique du schéma au démarrage : toute évolution de la structure de la base serait alors tracée, revue et réversible, ce qui est indispensable dès lors que des données réelles sont en jeu.

Enfin, si le périmètre fonctionnel s'élargissait — facturation, tiers payant, notifications aux patients — l'API monolithique pourrait être découpée en services plus fins, au prix d'une complexité accrue à mettre en balance avec les bénéfices attendus.

6. Analyse critique des limites

La limite la plus notable de mon périmètre tient au compromis réseau déjà évoqué : l'accès public restreint de la base, bien que sécurisé par un pare-feu et par TLS, reste moins satisfaisant qu'une isolation réseau complète, et ne conviendrait pas en l'état à un hébergement de données de santé réglementé.

La création du schéma au démarrage de l'application constitue une seconde limite : pratique pour un MVP, elle devient risquée en exploitation, où toute modification de structure doit être maîtrisée. De même, le dimensionnement en machines « burstable » plafonne la capacité sous une charge soutenue et prolongée ; ce choix, parfaitement adapté à une démonstration, devrait être réévalué pour un usage réel et continu par le cabinet.

Enfin, la couverture de tests automatisés de l'application, bien que présente et exécutée à chaque intégration, gagnerait à être étendue : les scénarios nominaux sont couverts, mais les cas d'erreur, les cas limites et les tests de charge mériteraient d'être développés pour fiabiliser davantage le service avant une ouverture à des utilisateurs réels.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Prérequis et configuration

Avant tout déploiement, l'équipe renseigne dans les secrets du dépôt les identifiants Azure (identifiant, mot de passe, tenant, abonnement) ainsi que l'adresse de courriel destinée aux alertes budgétaires. Ces informations, chiffrées par GitHub, ne sont jamais visibles dans le code ni dans les journaux d'exécution. Aucune autre préparation n'est nécessaire : le compte de stockage de l'état Terraform est créé automatiquement lors du premier lancement.

7.2 Déployer l'environnement

Le déploiement se lance depuis l'onglet Actions du dépôt, ou automatiquement par un envoi de code sur la branche principale. La chaîne provisionne l'infrastructure, construit et publie les images, puis déploie l'application. À la fin de l'exécution, l'adresse publique d'accès est affichée. L'ensemble ne demande aucune intervention manuelle intermédiaire.

7.3 Utiliser l'application

L'application est accessible à l'adresse publique de l'Ingress. L'interface web permet de gérer les patients ; la documentation interactive de l'API, utile pour les tests ou une future intégration avec d'autres logiciels du cabinet, est publiée sous le chemin « /api/docs ». Les points « /healthz » et « /readyz » permettent de vérifier en une commande que le service et sa base répondent correctement.

7.4 Reconstruire ou détruire

L'ensemble de l'infrastructure peut être reconstruit à l'identique par une seule action, ou détruit en choisissant l'action de destruction du workflow, ce qui interrompt immédiatement toute facturation. Cette reproductibilité garantit que le projet est duplicable et réutilisable, comme l'exige le cahier des charges.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de passer d'une logique de « code qui fonctionne sur ma machine » à une logique d'infrastructure et de déploiement entièrement automatisés et reproductibles. Cela m'a demandé de raisonner en permanence sur les cas d'échec et les états intermédiaires, et d'accepter plusieurs itérations avant d'obtenir une chaîne réellement fiable.

J'ai également dû apprendre à collaborer sur un périmètre partagé : nombre de problèmes que j'ai rencontrés se situaient à la frontière avec les responsabilités d'Elyess ou d'Anis, ce qui m'a appris à communiquer précisément et à documenter mes choix pour que l'équipe puisse s'appuyer dessus.

Forces

Je retiens comme force ma capacité à avoir une vision d'ensemble reliant le développement applicatif et son hébergement, ce qui m'a permis de concevoir une application pensée dès le départ pour être

déployée et exploitée, et non adaptée après coup.

J'ai aussi fait preuve de persévérance face aux difficultés d'industrialisation, en cherchant systématiquement la cause racine d'un incident plutôt qu'un contournement rapide, ce qui a bénéficié à la robustesse globale du projet.

Faiblesses

J'ai parfois eu tendance à vouloir déployer l'ensemble en une seule fois plutôt qu'à valider par petits incréments testés, ce qui a occasionné des allers-retours qui auraient pu être évités par une approche plus progressive.

Par ailleurs, mes compétences en sécurité réseau cloud, directement sollicitées par le choix d'hébergement de la base, restent perfectibles et constituent un domaine que je dois approfondir.

Compétences développées

Ce projet m'a permis de consolider ma maîtrise de Terraform et de la conception d'infrastructure Azure, de la conteneurisation Docker et du déploiement via Helm, ainsi que du développement d'une API moderne intégrée à une base de données relationnelle.

Au-delà des outils, j'ai acquis une compréhension concrète de l'articulation entre le code, l'infrastructure et l'exploitation, qui est au cœur du métier DevOps et que seule la réalisation d'un projet complet de bout en bout permet réellement d'intégrer.

Axes d'amélioration personnels

Pour de futurs projets, je souhaite adopter une approche plus incrémentale et testée des changements d'infrastructure, en validant chaque évolution isolément avant de l'intégrer à l'ensemble. Je compte également renforcer mes connaissances en réseau cloud — réseaux virtuels, points de terminaison privés, pare-feux applicatifs — afin de concevoir dès la première version des architectures plus sûres et plus proches des standards de production.